Organizing Agent Tools: From Tags to Registries

Imagine you're organizing a workshop. You wouldn't just throw all your tools into a big box—you'd want to organize them by purpose, perhaps keeping all the measuring tools in one drawer, cutting tools in another, and so on. This is exactly what we're doing with our agent tools using tags and registries. Let's explore how this organization system works and how it makes our lives easier when building agents.

Understanding Tool Organization

When we build agents, we often create many tools that serve different purposes. Some tools might handle file operations, others might work with databases, and still others might interact with external APIs. Our organization system has three layers:

1. Tool Decoration: Tag and document individual tools
2. Tool Registry: Central storage of all available tools
3. Action Registry: Curated sets of tools for specific agents

Let's see how these work together.

Tagging Tools for Organization

First, we use our decorator to tag tools based on their purpose:

```
@register_tool(tags=["file_operations"])
def read_file(file_path: str) -> str:
    """Reads and returns the content of a file."""
    with open(file_path, 'r') as f:
        return f.read()

@register_tool(tags=["file_operations", "write"])
def write_file(file_path: str, content: str) -> None:
    """Writes content to a file."""
    with open(file_path, 'w') as f:
        f.write(content)

@register_tool(tags=["database", "read"])
def query_database(query: str) -> list[dict]:
    """Executes a database query and returns results."""
    return db.execute(query)
```

When we register these tools, our decorator maintains two global registries:

- `tools`: A dictionary of all tools indexed by name
- `tools_by_tag`: A dictionary of tool names organized by tag

```
# Internal structure of tools_by_tag
{
    "file_operations": ["read_file", "write_file"],
    "write": ["write_file"],
    "database": ["query_database"],
    "read": ["query_database"]
}
```

This organization allows us to easily find related tools. For instance, we can find all tools related to file operations or all tools that perform read operations.

Creating Focused Action Registries

Now comes the powerful part. When we create an agent, we can easily build an `ActionRegistry` with just the tools it needs:

```
def create_file_processing_agent():
    # Create a registry with only file operation tools
    action_registry = ActionRegistry(tags=["file_operations"])

    return Agent(
        goals=[Goal(1, "File Processing", "Process project files")],
        agent_language=AgentFunctionCallingActionLanguage(),
        action_registry=action_registry,
        generate_response=generate_response,
        environment=Environment()
    )

def create_database_agent():
    # Create a registry with only database tools
    action_registry = ActionRegistry(tags=["database"])

    return Agent(
        goals=[Goal(1, "Database Operations", "Query database as needed")],
        agent_language=AgentFunctionCallingActionLanguage(),
        action_registry=action_registry,
        generate_response=generate_response,
        environment=Environment()
    )
```

Creating Specialized Agents

We can create agents with very specific tool sets just by specifying tags:

```
# Create an agent that can only read (no writing)
read_only_agent = Agent(
    goals=[Goal(1, "Read Only", "Read but don't modify data")],
    agent_language=AgentFunctionCallingActionLanguage(),
    action_registry=ActionRegistry(tags=["read"]),
    generate_response=generate_response,
    environment=Environment()
)

# Create an agent that handles all file operations
file_agent = Agent(
    goals=[Goal(1, "File Handler", "Manage file operations")],
    agent_language=AgentFunctionCallingActionLanguage(),
    action_registry=ActionRegistry(tags=["file_operations"]),
    generate_response=generate_response,
    environment=Environment()
)
```